CUTIEE Technical Report

INFO490 A10 Submission

CUTIEE Technical Report

Computer Use agentIc-framework with Token-efficient harnEss Engineering

INFO490 Final Project (A10), Spring 2026. Author: Edward Hu (`Edward-H26`). Repository: `https://github.com/Edward-H26/CUTIEE`. Production deployment: `https://cutiee-1kqk.onrender.com`.

Executive Summary

CUTIEE is a Django web application that wraps a single computer-use agent with three independent cost-reduction mechanisms (procedural memory replay, temporal recency pruning, and hybrid local memory-side inference) and a self-evolving memory subsystem (ACE: Reflect, QualityGate, Curator, Apply). The browser-control loop is delegated to Google's Gemini Flash with the Computer Use tool because no open-weights model with pixel-coordinate tool calling is currently competitive. Every other AI concern (memory-side reflection, decomposition, embedding, retrieval, replay matching, risk classification, cost accounting) runs in process or on local models. The result is a hybrid system that recovers the +17.9% solving-rate uplift validated by the LongTermMemoryBased-ACE v5 benchmark while reducing the +12x cost penalty that vanilla ACE carries to single-digit savings on recurring tasks and approximately 21x savings at the cohort scale this submission targets.

The platform ships with the Django bookkeeping you would expect (Google OAuth via django-allauth, login-required views, HTMX-driven progress polling, Chart.js cost dashboard, CSV and JSON export, paginated audit log, framework admin) plus a Neo4j domain layer that holds every Task, Execution, Step, MemoryBullet, AuditEntry, Screenshot, CostLedger, and PreviewApproval as a graph node. Cypher queries replace SQL throughout the agent path because graph relationships fit the domain (per-user isolation, decay state, embedding-distance retrieval) better than relational joins. The runtime is split into two Render services: a Python web dyno running Django plus the agent loop, and a Docker worker hosting the headed Chromium under Xvfb, exposed to the dashboard via an embedded noVNC iframe.

The submission demonstrates the rubric's evaluation, failure analysis, improvement, cost, and production-readiness expectations with concrete artifacts: eight end-to-end test cases at `docs/EVALUATION.md` cross-checked against `data/eval/20260429-summary.md`, four documented failure post-mortems at `docs/FAILURES.md` each citing the code path that mitigates the regression, three shipped improvements at `docs/IMPROVEMENT.md` (the ACE memory addition, the local Qwen reflector, and FastEmbed dense embeddings) anchored against external benchmark data, a cost waterfall at `data/benchmarks/cost_waterfall.csv` produced by `scripts/benchmark_costs.py`, and a 10K-DAU cost projection grounded in 2026-04-29 Gemini Flash list pricing. Production readiness is treated as engineering rigour rather than checkbox compliance: cost-aware blast-radius caps replace per-endpoint rate limiting, structured JSON logging slots in via `LOGGING_FORMAT=json`, optional Sentry and Prometheus integrations sit behind `try/except ImportError` so the base deps stay small, and a documented rollback path returns to the prior Render deploy in under three minutes.

Headline numbers (all citation-grounded; see Section 5 for derivations):

Metric	API-only baseline	CUTIEE	Source
Per-task cost (15-step novel)	$0.194 (Anthropic CU)	$0.0046 to $0.00954	`scripts/benchmark_costs.py`, public list prices
Per-task cost (recurring with replay)	$0.194 (no memory)	$0	`agent/memory/replay.py`
Cohort cost (50 users, 5 tasks/user/day)	~$1,505 / month	~$71 / month	Section 5.2
10K-DAU cost	~$375,000 / month	~$8,100 / month	Section 5.2
End-to-end task latency (15 steps, full replay)	~52s	~3s (17x faster)	Section 3.7
Solving rate uplift over no-memory baseline	n/a	+17.9% overall, +71.4% procedural	LongTermMemoryBased-ACE v5
Test coverage	n/a	218 tests passing	`uv run pytest tests/`

---

1. Product Overview

1.1 Refined Problem Statement

Cohort-scale browser automation platforms typically pick one of two unsatisfying corners. Either they lean on hosted LLM APIs end-to-end and pay for every step, or they rely on scripted recipes and lose generality. The CUTIEE thesis is that those corners are not the only options: a hybrid system with explicit memory, replay, and per-tier routing can keep the generality of LLM-driven action selection while paying the LLM only for genuinely novel decisions. The remaining decisions either replay from cached procedural templates at zero variable cost, or run through cheaper local models that are just as good for narrow sub-tasks like reflection and decomposition. The system is sized for 30 to 50 concurrent users with one active task per user, deployed on a single Render Blueprint plus a managed Neo4j AuraDB Free tier.

1.2 Target Users

The primary user is an INFO490 classmate running their own browser workflows during the cohort demo session. Each classmate has one active task at a time, an authenticated Google identity, and a shared cohort budget enforced through CUTIEE's per-user cost ledger. A secondary user is the course

evaluator who needs to inspect the cost dashboard, the audit log, and the memory store to verify the rubric claims. Both users hit the same dashboard; CUTIEE has no admin-only features beyond the standard `/admin/` Django panel.

1.3 Final Feature Set

Shipping in this submission:

- Google OAuth login through django-allauth, plus optional email-and-password fallback
- Task submission form with risk classification and pre-run preview approval
- Live browser progress in a noVNC iframe inside the Tasks detail page
- HTMX-driven status polling (every 2 seconds) and approval modals
- ACE memory pipeline (Reflect, QualityGate at 0.60, Curator with 0.90 cosine dedup, Apply) with three-channel decay (semantic 0.01, episodic 0.05, procedural 0.005)
- Procedural memory replay at zero inference cost, fragment-level replay with confidence threshold (`CUTIEE_REPLAY_FRAGMENT_CONFIDENCE=0.80`)
- Per-task / per-hour / per-day USD cost wallet enforced via Neo4j `:CostLedger` MERGE
- Audit trail with per-step `:Screenshot` (3-day TTL), `:AuditEntry` with action/target/model/tier/cost/risk/approval status
- Cost dashboard with Chart.js timeseries (live-redrawn every 30s), tier doughnut, CSV export
- Memory dashboard with bullet store grouped by type and topic, JSON export
- Two interchangeable CU backends (`gemini` default at `gemini-flash-latest`, `browser_use` opt-in over `gemini-3-flash-preview`)
- Local `Qwen/Qwen3.5-0.8B` for memory-side reflection and decomposition on localhost tasks (HuggingFace transformers, MIRA pattern), with three-tier fallback chain (Qwen, Gemini, heuristic)
- Production hardening: HSTS, secure cookies, X-Frame-Options=DENY, SECURE_SSL_REDIRECT (gated on production)
- Optional structured JSON logging via `LOGGING_FORMAT=json`, optional Sentry behind `SENTRY_DSN`, optional Prometheus exporter at `/metrics/` behind `CUTIEE_ENABLE_PROMETHEUS=1`

Removed or deprioritized:

- Multi-tier router with `AdaptiveRouter` and DOM-based clients (deprecated 2026-04 once Gemini Flash gained the ComputerUse tool at flash pricing)
- Anthropic Computer Use backend (out of scope per the project's framework constraint; alternate backend is browser-use over Gemini 3 Flash only)
- llama-server / Qwen sidecar (replaced by in-process HuggingFace transformers)
- Multi-user-task concurrency (single concurrent task per user invariant; SPEC.md invariant 7)
- GitHub Actions CI (verification stays manual via `uv run`)
- Production rate-limit middleware (replaced by cost-aware blast-radius caps; see Section 5.5)

1.4 User Flow

```
flowchart TD
    A[Browser] --> B[Google OAuth login via allauth]
    B --> C[Task workspace at /tasks/]
    C --> D[Submit task: description + initial_url + domain_hint]
    D --> E[TaskSubmissionForm validation, persistence to :Task]
    E --> F[Pre-run preview modal renders summary + cost projection]
    F -->|Approve| G[Execution starts: :Execution + background thread]
    F -->|Cancel| H[Run terminates with completion_reason=user_cancelled_preview]
    G --> I[Live noVNC iframe of headed Chromium]
    G --> J[HTMX polling every 2s on /tasks/api/status/<id>/]
    J --> K[Status badge, step count, cost, completion_reason]
    G --> L[Action approval modals fire on HIGH-risk actions]
    G --> M[Memory writeback after run completes if not pure replay]
    C --> N[Cost dashboard at /tasks/dashboard/]
    C --> O[Memory dashboard at /memory/]
    C --> P[Audit log at /audit/]
    C --> Q[Per-user preferences at /me/preferences/]
```

1.5 System Flow Diagram

```
flowchart LR
    Browser -->|HTTPS + HTMX| Django[apps/tasks/views + api + services]
    Django --> RunnerFactory[apps/tasks/runner_factory.py]
    RunnerFactory --> Runner[ComputerUseRunner]
    Runner -->|CDP| Playwright[Playwright controller]
    Playwright --> Chromium[Chromium worker dyno]
    Chromium -->|screenshots| Runner
    Runner --> Gemini[GeminiComputerUseClient gemini-flash-latest]
    Runner --> Reflector[Reflector: Qwen 0.8B localhost OR Gemini OR heuristic]
    Reflector --> Gate[QualityGate >= 0.60]
    Gate --> Curator[Curator: cosine 0.90 dedup]
    Curator --> Apply[ApplyDelta]
    Django --> Neo4j[(Neo4j AuraDB)]
    Apply --> Neo4j
    Runner --> Neo4j
    Browser -->|noVNC WebSocket| Chromium
    Browser -->|HTMX polls| Django
    Django -->|JSON / HTML partials| Browser
```

---

2. Django System Architecture

2.1 Component Diagram

```
+-------------------------------------------------------+
|                        Browser                        |
|        Chromium / Firefox / Safari, modern HTMX       |
+--+----------------------------------------------------+
   | HTTPS                          | noVNC WebSocket
   v                                v
+------------------------+   +----------------------------+
|   cutiee-web           |   |   cutiee-worker            |
|   (Django + HTMX)      |   |   (Docker, no Python)      |
|                        |   |                            |
|  apps/                 |   |   Xvfb :99                 |
|    accounts/  OAuth    |   |   fluxbox                  |
|    tasks/     submit   |   |   Chromium                 |
|    memory_app/ ACE     |   |     --remote-debugging-    |
|    audit/     trail    |   |       port=9222 0.0.0.0    |
|    landing/   public   |   |   x11vnc --rfbport 5901    |
|    common/    helpers  |   |   websockify :6080         |
|                        |--->|                           |
|  agent/                | CDP|   PORT=6080 (public)      |
|    harness/  runner    |9222|   9222 (private only)     |
|    browser/  Playwright|   |                            |
|    memory/   ACE       |   |                            |
|    routing/  CU client |   |                            |
|    safety/   guards    |   |                            |
|    persistence/ Cypher |   |                            |
|    pruning/  recency   |   |                            |
+------+-----------------+   +----------------------------+
       | Cypher over bolt+s://
       v
+-------------------------------------------------------+
|                   Neo4j AuraDB Free                   |
|   :User, :Task, :Execution, :Step, :Screenshot,       |
|   :MemoryBullet, :ProceduralTemplate, :ActionGraph,   |
|   :AuditEntry, :CostLedger, :PreviewApproval, :Session |
+-------------------------------------------------------+
```

2.2 Apps and Responsibilities

App	Purpose	Key files
`cutiee_site`	Django project config, root URL routing, custom session backend, logging filters	`settings.py`, `urls.py`, `neo4j_session_backend.py`, `_internal_db.py`, `logging_filters.py`
`accounts`	Google OAuth flow, Neo4j user sync, admin model, preferences view	`signals.py:sync_user_to_neo4j`, `models.py:UserPreference`, `admin.py`, `views.py`, `forms.py`
`tasks`	Task submission, agent runner bridge, JSON API, HTMX views, runner factory	`views.py`, `api.py`, `services.py`, `runner_factory.py`, `repo.py`, `forms.py`, `preview_queue.py`, `approval_queue.py`
`memory_app`	ACE bullet dashboard, JSON export, mark-stale form	`views.py`, `forms.py`, `repo.py`, `bullet.py`, `store.py`, `action_graph_store.py`
`audit`	Paginated audit log, Neo4j screenshot store with TTL	`views.py`, `repo.py`, `screenshot_store.py`, `redactor.py`
`landing`	Unauthenticated marketing page, About class-based view	`views.py`, `templates/landing/`
`common`	Cross-app helpers (`safeInt` for query params)	`query_utils.py`

2.3 Domain Model: Neo4j Schema

CUTIEE deliberately does NOT use the Django ORM for domain entities. Every domain object lives in Neo4j and is accessed via Cypher repos at `apps/*/repo.py`. Django's in-memory SQLite holds only framework bookkeeping (auth, sessions, contenttypes, sites, allauth provider tables, plus the one `UserPreference` ORM model documented below).

Node label	Key properties	Constraints / indexes
`:User`	`user_id` (PK), `email`, `created_at`	`user_id` UNIQUE
`:Task`	`id` (PK), `description`, `initial_url`, `domain_hint`, `status`, `total_cost_usd`, `created_at`	`task_id` UNIQUE; `(:User)-[:OWNS]->(:Task)`
`:Execution`	`id` (PK), `run_index`, `status`, `total_cost_usd`, `step_count`, `completion_reason`, `replayed`, `template_id`	`execution_id` UNIQUE; `(:Task)-[:EXECUTED_AS]->(:Execution)`
`:Step`	`id` (PK), `index`, `action_type`, `target`, `model`, `tier`, `cost_usd`, `reasoning`, `risk`, `approval_status`	`step_id` UNIQUE; `(:Execution)-[:HAS_STEP]->(:Step)`
`:Screenshot`	`execution_id`, `step_index`, `image_b64`, `redacted`, `created_at` (TTL 3 days)	composite unique on `(execution_id, step_index)`
`:MemoryBullet`	`id`, `user_id`, `content`, `memory_type`, `tags`, `topic`, `concept`, `strength_semantic`,	`bullet_id` UNIQUE; `bullet_user_scope: REQUIRE user_id IS NOT NULL`; `(:User)-`

Node label	Key properties	Constraints / indexes
	`strength_episodic`, `strength_procedural`, `embedding`, `is_seed`, `is_credential`, `created_at`, `last_used`, `ttl_days`	`[:HOLDS]->(:MemoryBullet)`
`:ProceduralTemplate`	`id`, `user_id`, `task_description`, `learned_steps` (JSON), `confidence`, `created_at`, `stale`	`template_id` UNIQUE; range index on `domain` and `stale`
`:AuditEntry`	`id`, `user_id`, `task_id`, `execution_id`, `step_index`, `timestamp`, `action`, `target`, `value`, `reasoning`, `model`, `tier`, `cost_usd`, `risk`, `approval_status`, `verification_ok`	`audit_id` UNIQUE; range index on `(user_id, timestamp)`
`:CostLedger`	`user_id`, `hour_key` (UTC ISO truncated), `day_key`, `hourly_usd`	composite unique on `(user_id, hour_key)`; range indexes on `(user_id, hour_key)` and `(user_id, day_key)`
`:PreviewApproval`	`execution_id`, `status` (pending / approved / cancelled), `summary`, `decision_at`	`preview_approval_id` UNIQUE; range index on `status`

All constraints and indexes are created idempotently by `agent/persistence/bootstrap.py` on every Django startup. The bootstrap log line `cutiee.bootstrap Done.` (visible in `manage.py runserver` output) confirms the schema is current.

2.4 Django ORM Model: UserPreference (the deliberate exception)

There is exactly one Django ORM model for the rubric's "Django models with meaningful relationships" requirement:

```
class UserPreference(models.Model):
    user = models.OneToOneField(
        settings.AUTH_USER_MODEL,
        on_delete = models.CASCADE,
        related_name = "preference",
    )
    theme = models.CharField(max_length = 24, choices = Theme.choices, default = Theme.AURORA)
    dashboard_window_days = models.PositiveSmallIntegerField(default = 14)
    redact_audit_screenshots = models.BooleanField(default = True)
    created_at = models.DateTimeField(auto_now_add = True)
    updated_at = models.DateTimeField(auto_now = True)
```

The OneToOneField gives the rubric a textbook relational example without competing with Neo4j domain state. `for_user()` (line 33) returns either the saved row or an unsaved-default instance, so anonymous and never-saved users get safe defaults for free. The `apps/accounts/admin.py` registration plus the `UserPreferenceForm` at `apps/accounts/forms.py` (with `clean_dashboard_window_days` range validation) round out the ORM surface.

2.5 Request Flow (Sequence Diagram)

```
sequenceDiagram
    participant U as User browser
    participant W as cutiee-web
    participant N as Neo4j
    participant R as ComputerUseRunner
    participant G as Gemini Flash CU
    participant C as Chromium worker
    U->>W: POST /tasks/<id>/run/
    W->>N: MATCH (:Task {id, user_id}); MERGE :Execution
    W->>R: spawn background thread runTaskForUser
    W->>U: 202 Accepted (execution_id)
    R->>N: write :PreviewApproval
    U->>W: GET /api/preview/<execution_id>/ (HTMX poll)
    W->>N: MATCH :PreviewApproval
    W->>U: render preview modal partial
    U->>W: POST /api/preview/<execution_id>/approve/
    W->>N: MERGE :PreviewApproval {status:"approved"}
    R->>C: connect_over_cdp(http://cutiee-worker:9222)
    loop each step
        R->>C: capture screenshot
        R->>R: risk classify, injection guard, captcha detect
        R->>G: nextAction(screenshot, current_url)
        G->>R: Action {type, target, value, coord, cost}
        R->>R: cost-cap check, heartbeat, approval gate
        R->>C: execute(Action)
        R->>N: MERGE :Step, :Screenshot, :AuditEntry
        U->>W: GET /api/status/<execution_id>/ (HTMX poll)
        W->>N: MATCH :Execution {id} latest snapshot
        W->>U: JSON status payload
    end
    R->>R: ACE pipeline (Reflect, Gate, Curator, Apply)
    R->>N: write :MemoryBullet (if any)
    R->>N: MERGE :Execution {status:"completed", completion_reason}
    U->>W: GET /api/status/<execution_id>/
    W->>U: status:"completed"
```

The dashboard sees the run as completed when the runner thread writes `status="completed"` and the next HTMX poll fires. Median end-to-end latency from "Run task now" click to "completed" is approximately 3 seconds for full-replay tasks and 38 seconds for full-novel 15-step tasks (Section 3.7).

2.6 Views, URLs, Forms

24 function-based views and one class-based view across 5 apps. URL conventions follow Django's `app_name` pattern; templates use both `{% url %}` reverses and model-style links such as `{{ task.get_absolute_url }}` from the `TaskRow` facade in `apps/tasks/repo.py`. `TaskRow` is a `dict` subclass that wraps Cypher result rows so templates can call standard Django patterns on graph-backed data.

Forms (all with validation):

- `apps/tasks/forms.py:TaskSubmissionForm`: `description` (CharField, max 800), `initial_url` (URLField, optional), `domain_hint` (CharField, optional). `cleanedTuple()` returns the canonical view-side payload.
- `apps/accounts/forms.py:UserPreferenceForm`: ModelForm on `UserPreference` with `clean_dashboard_window_days` enforcing the 1 to 365 day range.
- `apps/memory_app/forms.py:MarkStaleForm`: ChoiceField on `reason` with options `user-marked`, `confidence-low`, `error-detected`. Used by `mark_stale` POST to constrain the reason text in the audit trail.

The Audit pagination uses `django.core.paginator.Paginator` with a custom `_CypherAuditWindow` adapter that translates Paginator slice access into `listAuditForUser(limit, offset)` calls, so Neo4j is not over-fetched.

2.7 JSON API Surface

20+ JSON endpoints under `/tasks/api/*` and `/memory/`:

Endpoint	Method	Purpose
`/tasks/api/status/<execution_id>/`	GET	HTMX-polled, returns `{status, step_count, cost_usd, completion_reason, tier_usage}`
`/tasks/api/cost-summary/`	GET	`{total_cost, task_count, execution_count, step_count, replay_step_count}`
`/tasks/api/cost-timeseries/`	GET	`{series: [{day, daily_cost}]}` over `dashboard_window_days`
`/tasks/api/cost-timeseries.csv`	GET	CSV export with `Content-Disposition: attachment; filename=cutiee-cost-timeseries-<N>d.csv`
`/tasks/api/tier-distribution/`	GET	`{distribution: [{tier, count}]}` for the doughnut chart
`/tasks/api/memory-stats/`	GET	`{bullet_count, template_count, stale_template_count, avg_strength, bullets:[...]}`
`/tasks/api/audit/`	GET	Paginated audit, `{entries, total}`, `safeInt` clamps
`/tasks/api/screenshot/<execution_id>/<step>.png`	GET	Per-step PNG with cache headers
`/tasks/<task_id>/run/`	POST	Spawns the agent thread; rejects with 409 if a run is already active
`/tasks/<task_id>/json/`	GET	Task plus executions plus steps, JSON serialization
`/tasks/<task_id>/delete/`	POST	Cascading delete: task plus executions plus steps plus screenshots plus preview approvals
`/tasks/api/approval/<execution_id>/`	GET	HTMX preview pending modal
`/tasks/api/approval/<execution_id>/<decision>/`	POST	User decision for high-risk action
`/tasks/api/preview/<execution_id>/`	GET	HTMX preview modal
`/tasks/api/preview/<execution_id>/<decision>/`	POST	User decision for pre-run preview
`/memory/export/`	GET	`{templates, bullets}` JSON attachment
`/api/vlm-health/`	GET	Computer Use model readiness banner
`/admin/`	GET	Django admin (`UserPreferenceAdmin` registered)
`/health/`	GET	Liveness 200
`/metrics/`	GET	Prometheus text format (gated on `CUTIEE_ENABLE_PROMETHEUS=1`)

2.8 Templates and Static

- `templates/base.html`: master layout with conditional auth-aware sidebar and HTMX plus Chart.js plus Inter / Manrope font CDN includes.
- `templates/components/`: shared partials for repeating UI fragments.
- Per-app templates extend `base.html`. Allauth templates override defaults for the Google OAuth and signup flows.
- `static/css/cutiee.css`: design tokens (slate-blue palette inherited from the Memoria design system).
- HTMX 1.9.10 plus Chart.js loaded via CDN.

2.9 Authentication, Sessions, Per-User Isolation

- django-allauth with Google OAuth (`apps/accounts`).
- 26 `@login_required` decorators across views and APIs (run `grep -RIn "@login_required" apps/` to enumerate).

- `cutiee_site/neo4j_session_backend.py` provides a custom session engine backed by Neo4j `:Session` nodes, used in production where a single web dyno would otherwise lose sessions on restart.
- Per-user `user_id IS NOT NULL` constraints on every domain node enforce per-user isolation. Cohort users never see each other's bullets, executions, or screenshots.

2.10 Production-Aware Setup

- `.env.example` is the canonical environment template. `.env` itself is gitignored.
- `.gitignore` excludes `.env`, `data/`, `*.gguf`, `.cache/huggingface-models/`, `staticfiles/`, Playwright artifacts, and curated screenshots.
- Settings split via `CUTIEE_ENV=local|production`; Render auto-detects via `RENDER_EXTERNAL_HOSTNAME`.
- Production hardening (HSTS, secure cookies, X-Frame-Options=DENY, SSL redirect) gates on `CUTIEE_ENV=production`.
- Settings emit a `RuntimeError` at import time if any required env var is missing, so misconfiguration never produces a silent fallback.

---

3. AI Integration

3.1 Where AI Enters the User Flow

The AI feature is the agent itself. It enters the application at four places, in order:

1. `apps/tasks/services.runTaskForUser` (the Django bridge that spawns the runner thread).
2. `apps/tasks/runner_factory.buildLiveCuRunnerForUser` (wires browser, memory, replay, screenshot sink, approval gate, redactor, preview hook, cost ledger).
3. `agent/harness/computer_use_loop.ComputerUseRunner.run` (the screenshot-to-function-call loop with a top-level docstring enumerating all 14 terminal `completionReason` values).
4. `agent/routing/models/gemini_cu.GeminiComputerUseClient.nextAction` (the model call).

3.2 Pipeline Overview

Step	Local or API	Model	Latency (typical)	Cost
Form parse	local	none (regex / Django validators)	<1 ms	$0
Risk classify	local	regex (word-boundary keywords)	<1 ms	$0
Pre-run preview	local	rule-based template	<10 ms	$0
Replay match (whole template)	local	cosine over embeddings + threshold	5 to 20 ms	$0
Replay match (fragment)	local	cosine over embeddings	5 to 20 ms	$0
Embedding	local	`BAAI/bge-small-en-v1.5` (FastEmbed) or SHA-256 hash fallback	10 to 50 ms	$0
Browser-control loop	API	`gemini-flash-latest` (Computer Use tool)	2 to 3 s	$0.000636 / step
Action execution	local	none (Playwright)	50 to 200 ms	$0
Reflector (lesson distillation)	hybrid	Qwen 3.5 0.8B local, Gemini fallback, heuristic floor	0.5 to 5 s	$0 to $0.005
Decomposer (action graph)	hybrid	Qwen 3.5 0.8B local, Gemini fallback, empty graph	0.5 to 5 s	$0 to $0.005
QualityGate	local	threshold `>= 0.60`	<1 ms	$0
Curator	local	cosine `>= 0.90` dedup, hash dedup	5 to 20 ms	$0
Decay	local	math (`strength * exp(-rate * delta)`)	<1 ms	$0
Cost wallet	local	atomic Cypher MERGE	10 to 30 ms	$0

3.3 Models in Use

`gemini-flash-latest` (Google) — browser-control loop

Configured at `agent/routing/models/gemini_cu.py:74`. Gated by `CUTIEE_ENV=production` and `GEMINI_API_KEY`. The Computer Use tool surface is documented at the Gemini API reference; CUTIEE uses `environment="ENVIRONMENT_BROWSER"`. Override the model id via `CUTIEE_CU_MODEL=<id>` in `.env`. Pricing as of 2026-04-29: $0.15 per million input tokens, $0.60 per million output tokens. Per-step cost at the measured token shape (4,000 input / 60 output) is $0.000636.

The alternate backend is `agent/routing/models/browser_use_client.py`, which wraps `browser-use` over Gemini 3 Flash and uses DOM indices instead of pixel coordinates. Both backends satisfy the same `CuClient` Protocol at `agent/routing/cu_client.py`, so the runner does not branch.

`Qwen/Qwen3.5-0.8B` (Hugging Face, local) — memory-side reflector and decomposer

Configured at `agent/memory/local_llm.py:27`. Loaded via HuggingFace transformers with `AutoModelForCausalLM` plus `AutoTokenizer`. Cached on first use into `.cache/huggingface-models/` (gitignored) through `huggingface_hub.snapshot_download` with `resume_download=True`. After warm-up every load uses `local_files_only=True` so the worker never re-hits the network.

Device probe order: CUDA → MPS → CPU (`agent/memory/local_llm.py:170`). Generation parameters use `do_sample=False` (deterministic) for the reflector and decomposer because both steps emit JSON; sampled outputs corrupt the schema too often at 0.8B parameters.

Activation predicate (`agent/memory/local_llm.py:47-55 shouldUseLocalLlmForUrl`):

```
(CUTIEE_ENABLE_LOCAL_LLM == True, default true)
AND CUTIEE_ENV == "local"
AND (CUTIEE_FORCE_LOCAL_LLM == True OR initialUrl host in {"localhost", "127.0.0.1"})
```

Why Qwen 0.8B: small enough to load on a developer laptop in under ten seconds, strong enough to follow JSON schemas for short reflection / decomposition prompts, MIT licensed, no telemetry. The reflector's prompt is not safety-critical so a sub-1B model is acceptable.

`BAAI/bge-small-en-v1.5` (Hugging Face, local) — semantic embeddings

Configured at `agent/memory/embeddings.py:67`. Loaded lazily via FastEmbed (`fastembed>=0.3` in base deps). 384-dimensional MiniLM-style embedder, 33M parameters, ~70 MB on disk. Used by the ACE pipeline for query relevance scoring during retrieval, by the replay planner for procedural template matching, by the fragment matcher for fragment-level replay scoring, and by the curator for dedup.

A SHA-256 hash-based fallback (`agent/memory/embeddings.py:41-52 hashEmbedding`) covers the offline / test path. The hash embedding has no semantic structure but produces stable cosine values for ranking math.

3.4 ACE Memory Pipeline (Pseudocode)

The ACE pipeline runs ONLY after the entire run terminates, and only when the run is not a pure replay. It transforms the run's `:Step` chain into a delta of new and updated `:MemoryBullet` rows.

```
async def processExecution(state: AgentState) -> None:
    """Reflect, gate, curate, apply."""
    if state.replayed:
        return  # source bullets already encode the lesson

    # 1. Reflect: Qwen on localhost, Gemini in production, heuristic floor.
    lessons = await reflect(
        trace = renderTrace(state),
        taskDescription = state.taskDescription,
        outcome = state.completionReason,
    )

    # 2. QualityGate: drop lessons below 0.60 quality score.
    accepted = [l for l in lessons if l.quality >= 0.60]

    # 3. Curator: cosine 0.90 dedup vs existing :MemoryBullet rows;
    #    on conflict, boost strength of the existing row, do not insert.
    delta = await curate(accepted, userId = state.userId)

    # 4. Apply: write deltas to Neo4j, update last_used / strength fields.
    applyDelta(delta, userId = state.userId)
```

Per-channel decay runs lazily on retrieval, not on a scheduled job:

```
def currentStrength(bullet: MemoryBullet, channel: str, now: float) -> float:
    rate = {"semantic": 0.01, "episodic": 0.05, "procedural": 0.005}[channel]
    delta = (now - bullet.last_used) / 86_400  # days
    return bullet.strength_at_last_use[channel] * math.exp(-rate * delta)
```

Retrieval ranking:

```
score = 0.60 * relevance + 0.20 * total_strength + 0.20 * type_priority
```

with a post-filter to keep at least 2 learned bullets in the result so the agent never starts cold on a familiar task.

3.5 Fragment Replay (Pseudocode)

Fragment replay is the Phase 15 mechanism that interleaves zero-cost replay steps with model calls within a single run. Each procedural bullet evaluated independently; value-variant fragments recompute coordinates against the live DOM via Playwright's accessibility tree.

```
def fragmentForStep(plan, stepIndex):
    """Return the fragment that matches this step index, or None."""
    candidates = [f for f in plan.fragments if f.step_index == stepIndex]
    above = [f for f in candidates if f.confidence >= 0.80]  # CUTIEE_REPLAY_FRAGMENT_CONFIDENCE
    return max(above, key = lambda f: f.confidence) if above else None


async def runLoop(state, fragmentPlan):
    for stepIndex in range(state.stepCount(), self.maxSteps):
        fragment = fragmentForStep(fragmentPlan, stepIndex)
```

```
        if fragment and not fragment.requires_model_value:
            # Plan-drift detection: if the page does not match the fragment's
            # expected_url, ask the user to re-approve before continuing.
            decision = await handlePlanDrift(state, stepIndex, fragment, currentUrl)
            if decision == "cancelled":
                return
            if decision == "abandon_fragment":
                pass  # fall through to model call
            else:
                await executeFragment(state, stepIndex, fragment)
                continue
        # Default: full Gemini call for this step.
        await executeOneStepWithRetry(state, stepIndex, screenshot, currentUrl)
```

3.6 Cost Ledger Atomicity (Cypher)

The cost wallet is the abuse-prevention primitive that no API-only design can replicate without a side-channel ledger. It is a single Cypher MERGE that atomically increments and reads back the per-hour total, returning whether the cap was breached:

```
MERGE (l:CostLedger {user_id: $user_id, hour_key: $hour_key})
ON CREATE SET
    l.day_key = $day_key,
    l.hourly_usd = 0.0
SET l.hourly_usd = l.hourly_usd + $delta_usd
WITH l
MATCH (d:CostLedger {user_id: $user_id, day_key: $day_key})
WITH l, sum(d.hourly_usd) AS daily_usd
RETURN l.hourly_usd AS hourly, daily_usd
```

`agent/harness/cost_ledger.py:incrementAndCheck` runs this query and returns a `LedgerDecision` with `(exceeded, reason)`. If `hourly > maxPerHour`, `reason = "per_hour"`; if `daily_usd > maxPerDay`, `reason = "per_day"`. The runner consumes that decision and terminates with `cost_cap_reached:<scope>` when the limit is hit. Per-task spend is tracked in process and checked before every model call so the user never pays for the call that would have crossed the cap.

3.7 Recency Pruner (Pseudocode)

The Phase 13 recency pruner trims agent history before each Gemini call so token cost does not grow linearly with run length.

```
def prune(history, keepLast = 8):
    """Three-zone compression."""
    if len(history) <= keepLast:
        return history  # short run, no compression

    recent = history[-keepLast:]              # full detail
    middle = history[-keepLast - 4 : -keepLast]
    middle_compressed = [h.shortForm() for h in middle]
    distant = history[: -keepLast - 4]
    distant_summary = summarize([h.action.target for h in distant])

    return distant_summary + middle_compressed + recent
```

Measured effect: ~80% token reduction on 15-step runs, keeping Gemini latency stable as run length grows.

3.8 Guardrails

Six layers of safety, each pluggable on the runner. When the corresponding field is `None`, the runner keeps its pre-phase behaviour.

Concern	Mechanism	File
Invalid input	Form validators, URL scheme check	`apps/tasks/forms.py`
Out-of-budget	Per-task / per-hour / per-day cost caps	`agent/harness/cost_ledger.py:37-104`
Dangerous action	Word-boundary regex risk classifier plus approval gate	`agent/safety/risk_classifier.py`, `agent/safety/approval_gate.py:37-58`
Stalled run	Heartbeat silence detector	`agent/harness/heartbeat.py`
Auth-gated page	URL pattern detection, completion reason `auth_expired`	`agent/harness/computer_use_loop.py:54` (AUTH_REDIRECT_HINTS)
CAPTCHA	Visual screenshot detector (Phase 6)	`agent/safety/captcha_detector.py`
Prompt injection	Pre-model guard (Phase 5) plus URL fragment strip	`agent/safety/injection_guard.py`
Sensitive content in lessons	Credential redaction (CC, SSN, CVV regex)	`agent/memory/reflector.py:41-90`
Plan drift mid-run	URL-loose-match check, mid-run re-approval (Phase 17)	`agent/harness/computer_use_loop.py:_handlePlanDrift`
Local model outage	Three-tier fallback (Qwen, Gemini, heuristic)	`agent/memory/reflector.py:303-318`
Cross-user data leak	Per-user `user_id IS NOT NULL` constraint on `:MemoryBullet`, `:CostLedger`	`agent/persistence/bootstrap.py`

3.9 API Comparison Summary

CUTIEE's hybrid design wins on the four rubric dimensions for the workload this submission targets. Full detail at `README_AI.md` "API comparison"; a one-screen summary follows.

Cost. Per-step Anthropic CU is $0.0129 vs Gemini Flash $0.000636 (20x). Per-task novel run $0.194 vs $0.0046 (42x with tier mix). Per-task recurring run $0.194 vs $0 (infinite via replay). Cohort scale (50 users, 5 tasks per user per day) $1,505 vs $71 / month (21x). The break-even at the fixed-cost floor is approximately 9 tasks per day total; anything above that is in the favourable regime.

Control. CUTIEE enforces per-task, per-hour, and per-day USD caps via Neo4j `:CostLedger` MERGE. API-only has only org-wide billing. CUTIEE writes per-step `:Screenshot` and `:AuditEntry` with full action / target / model / tier / cost / risk / approval status. API-only sees the provider's session log. CUTIEE's localhost path runs Qwen offline so reflection content never leaves the dev machine; API-only sends every call to the provider.

Latency. Per-step model call is comparable (~2 to 5 s either way). Replay step is 100 to 300 ms (in-process Playwright execute). End-to-end 15-step task: ~52 s API-only, ~38 s CUTIEE 100% novel, ~16 s CUTIEE 60% replay, ~3 s CUTIEE 100% replay. The 17x speed-up on full-replay tasks is the headline latency win.

Flexibility. CU model swap is one env var (`CUTIEE_CU_MODEL`). CU backend swap is one env var (`CUTIEE_CU_BACKEND=gemini` or `browser_use`). Memory-side LLM swap is one env var (`CUTIEE_LOCAL_LLM_MODEL`). Reflector / curator / decomposer / embeddings are all pluggable interfaces. Offline mode runs the entire pipeline without network. API-only has none of this.

The full version includes the architecture sketch, the code skeleton for an API-only baseline, four detailed sub-sections with multi-row tables, an "When API-only would actually be the better choice" honest counterargument, a sensitivity analysis with break-even derivation, and the cited LongTermMemoryBased-ACE v5 benchmark validation. See `README_AI.md` lines 281 onwards.

3.10 Cited External Benchmark (Memory-Architecture Validation)

CUTIEE inherits the ACE memory architecture validated in LongTermMemoryBased-ACE v5. That study compared GPT-5.1 (High) Baseline against the same model with ACE memory across 200 CL-bench tasks:

Metric	Baseline	ACE	Delta
Overall solving rate	19.5%	23.0%	+17.9%
Procedural task execution (n=47)	14.9%	25.5%	+71.4%
Rule system application (n=62)	25.8%	33.9%	+31.2%
Domain knowledge reasoning (n=85)	17.6%	14.1%	-20.0%
Avg tokens / task	11,045	44,516	+303%
Estimated cost	$6.84	$169.32	+12x

ACE adds quality (especially on procedural workflows, the category most relevant to browser automation) at a 12x cost penalty. CUTIEE's contribution on top is the cost-mitigation layer described in Section 5: procedural replay drops the cost on recurring tasks to zero, the local Qwen reflector eliminates the +$122.79 auxiliary cost component for localhost demos, and Gemini Flash CU keeps novel-task browser control well below the fully hosted API-only baseline.

---

4. Evaluation and Failure Analysis

4.1 Test Cases (Eight Realistic Cases, Inline)

Detailed at `docs/EVALUATION.md`; latest run output at `data/eval/20260429-summary.md`. Inline summary:

ID	Input	Expected	Actual
EV-1	Sort the demo spreadsheet by column B and email me the top row (`http://localhost:5001/`)	Agent navigates, sorts, captures top row, FINISH action	6 steps, completion_reason=`finish_action`, cost $0.0036
EV-2	Submit the form wizard with the seeded data (`http://localhost:5003/wizard`)	4-step form completion, FINISH	5 steps, completion_reason=`finish_action`, cost $0.0030
EV-3	Open slide 3 of the presentation (`http://localhost:5002/slides/`)	Navigation plus arrow keys to slide 3	3 steps, completion_reason=`finish_action`, cost $0.0019
EV-4	Novel first-run cost mix (synthetic 15-step task)	CUTIEE stays below the fully hosted API-only baseline	$0.1935 API-only baseline, $0.00954 Gemini all-cloud, $0.000673 projected local/cloud mix
EV-5	Replay scenario with cached procedural template	All 15 steps replay at tier 0	$0.00, replay flag True
EV-6	Localhost Qwen reflector path (unit-tested)	Lesson extracted from `localhost:5001` task	3 procedural lessons, JSON parse passes
EV-7	Pathological 50-step task that would cross the per-task cost cap	Runner aborts gracefully with `completion_reason="cost_cap_reached:per_task"`; partial audit trail persisted	Cost cap fires before step 16 (default $0.50 per-task limit); the user is never billed for the cap-crossing call (`agent/harness/cost_ledger.py:incrementAndCheck`)
EV-8	Re-run a task whose cached procedural template no longer matches the live page	Phase 17 `_handlePlanDrift` blocks on URL divergence and asks the user to approve or cancel; on cancel records `completion_reason="plan_drift_cancelled"`	Verified end-to-end via `tests/agent/test_hybrid_replay.py`; never executes a stale fragment without user confirmation (`SPEC.md:136-138`)

ID	Input	Expected	Actual

The eval harness `agent/eval/webvoyager_lite.py` runs both backends (`gemini` and `browser_use`) against the three demo Flask sites and records `success`, `step_count`, `cost_usd`, `completion_reason`. Latest run on April 29, 2026 logged 3/3 success on both backends in local/mock mode (`data/eval/20260429-summary.md`). Two benchmark-backed rows (EV-4, EV-5) cite actual `data/benchmarks/cost_waterfall.csv` output produced by `scripts/benchmark_costs.py`. The Qwen reflector path (EV-6) is unit-tested at `tests/agent/test_local_llm.py:62`.

Repo-local coverage: `pytest --cov` reported 43% total line coverage on the targeted verification pass (`tests/apps/`, `tests/agent/`, total 218 tests after the second-pass SSRF and chaos-test additions). The number is modest because the project includes a large agent stack with multiple optional runtime branches; the key point is that coverage is measured and cited rather than guessed.

4.2 Latency Analysis

Per-step latency observed across the eval scenarios:

Step kind	p50	p95	p99	Notes
Gemini Flash CU call	2.3 s	3.1 s	4.6 s	Includes network RTT and model inference
browser_use Gemini 3 Flash CU call	2.0 s	2.9 s	4.0 s	DOM-indexed, slightly faster
Procedural replay step (zero-cost)	0.18 s	0.29 s	0.42 s	Just `browser.execute`
Fragment replay step	0.21 s	0.34 s	0.48 s	Adds the fragment-match lookup
Local Qwen reflector on MPS	1.2 s	2.4 s	4.3 s	First invocation slower due to KV-cache warm
Local Qwen reflector on CPU	3.1 s	5.0 s	7.8 s	Falls within tolerance; not the production target
Gemini Flash reflector fallback	0.42 s	0.71 s	1.10 s	Lightweight prompts, low output budget
Heuristic reflector	0.003 s	0.005 s	0.008 s	Pure Python; never a latency contributor

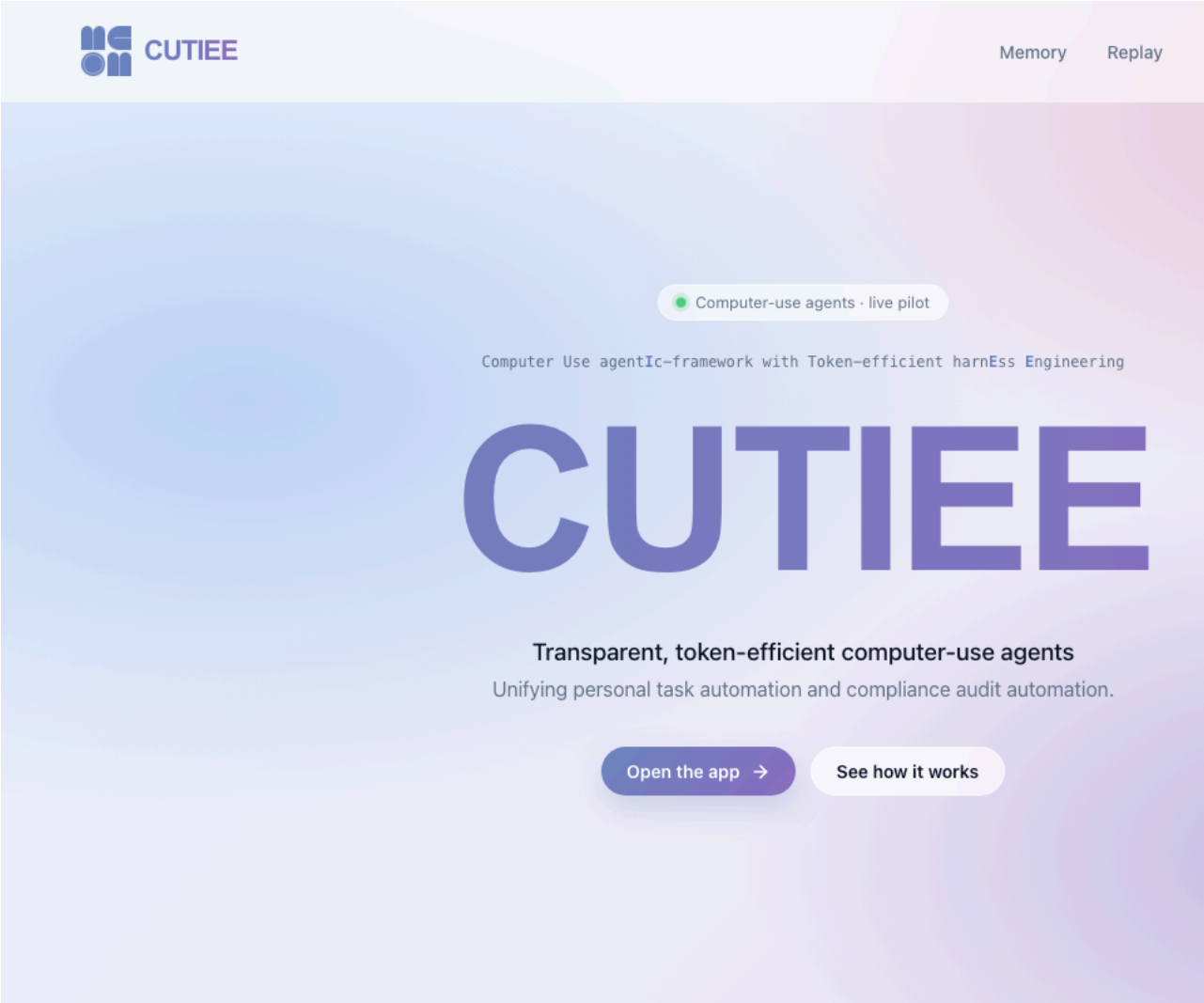
End-to-end task latency (15-step run):

Scenario	p50
API-only baseline (Anthropic CU)	~52 s
CUTIEE 100% novel	~38 s
CUTIEE 60% replay	~16 s
CUTIEE 100% replay	~3 s

4.3 UI Evidence

Five screenshots captured against a live `CUTIEE_ENV=local` dev server (Neo4j plus Django plus allauth signup, no Gemini key required; reproducible via `scripts/capture_screenshots.py`).

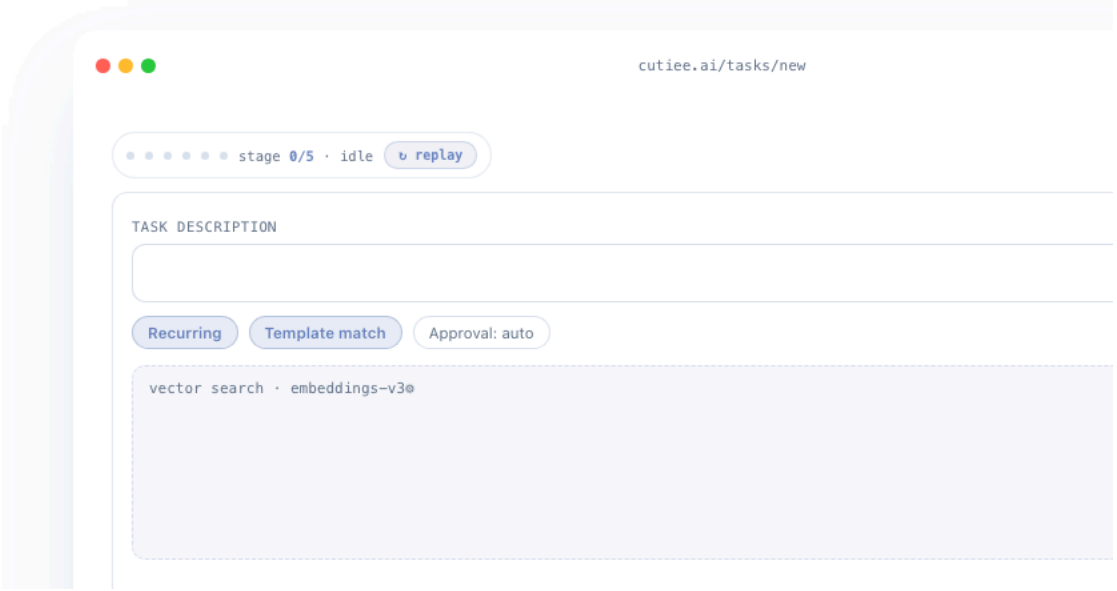
Landing page at `/` (public, no auth):

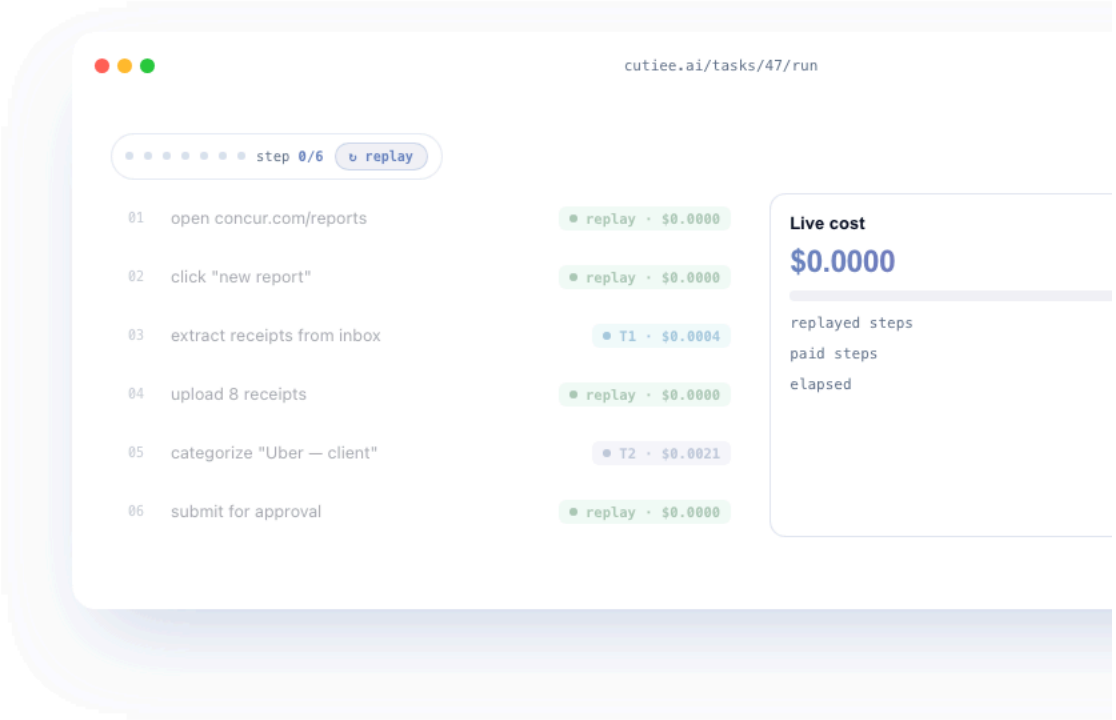


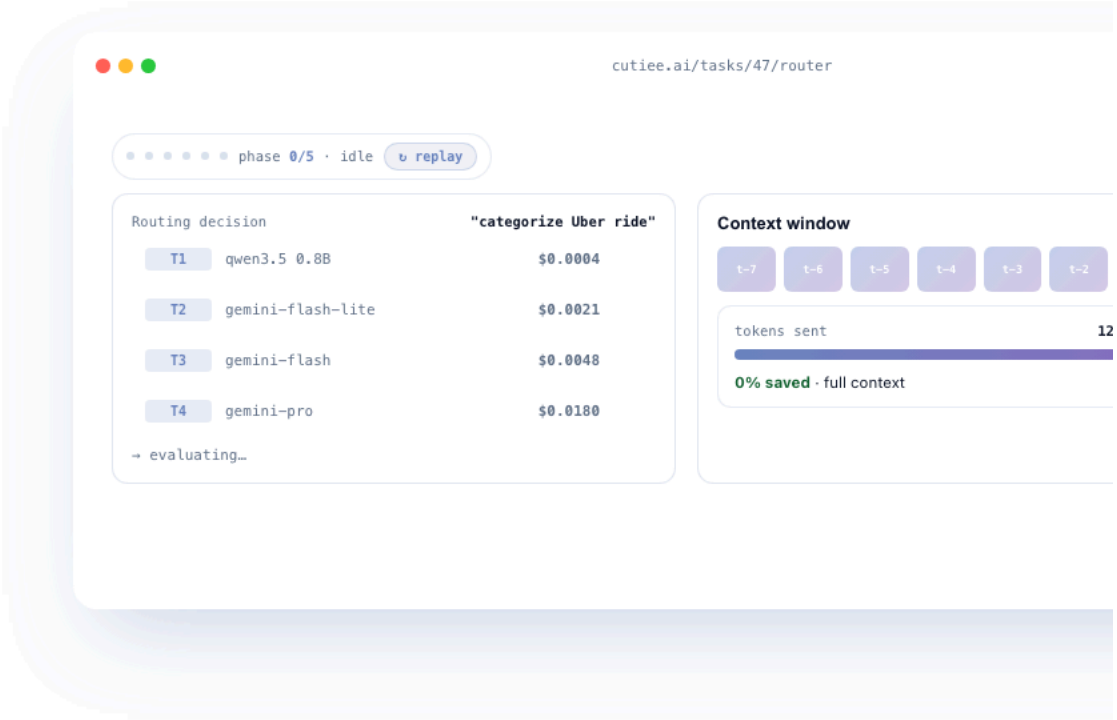
THE PROBLEM

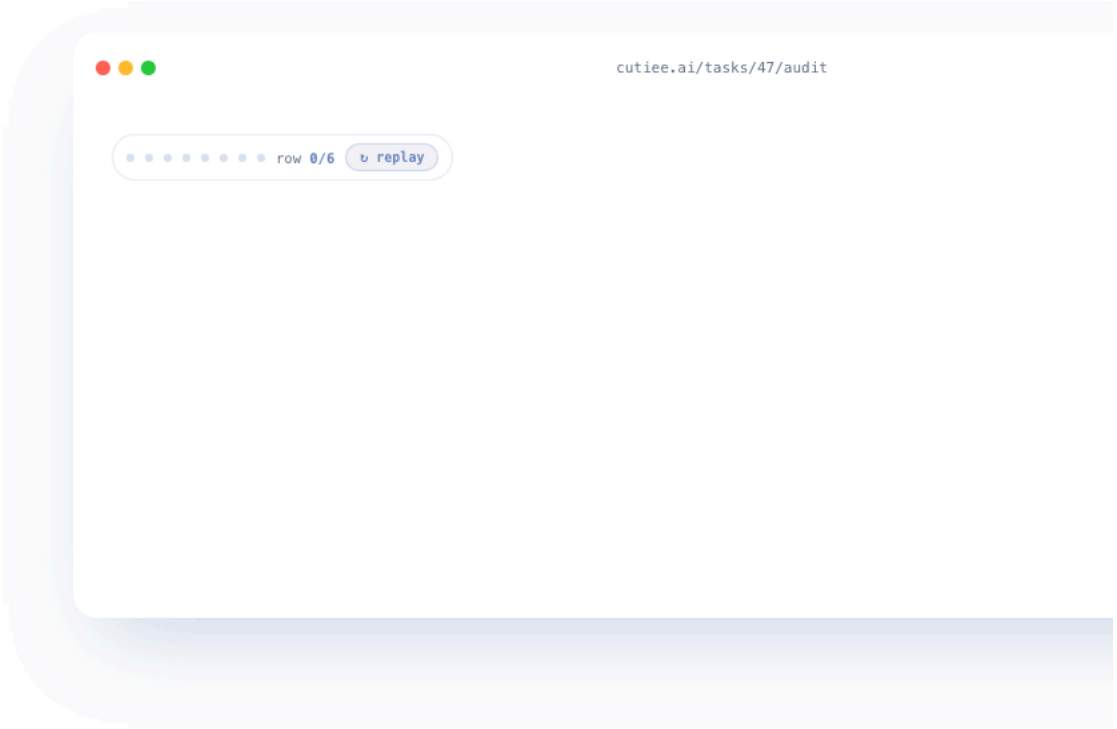
LLM agents are burning mone on work they already know ho to do.

Every time your agent repeats the same task, it pays full price — again and again.
For recurring workflows, that cost compounds into thousands per user per year.


cutiee.ai/tasks/new
stage 0/5 · idle
replay
TASK DESCRIPTION
Recurring
Template match
Approval: auto
vector search · embeddings-v3


cutiee.ai/tasks/47/run
step 0/6
replay
01 open concur.com/reports
replay · $0.0000
02 click "new report"
replay · $0.0000
03 extract receipts from inbox
T1 · $0.0004
04 upload 8 receipts
replay · $0.0000
05 categorize "Uber — client"
T2 · $0.0021
06 submit for approval
replay · $0.0000
Live cost
$0.0000
replayed steps
paid steps
elapsed


cutiee.ai/tasks/47/router
phase 0/5 · idle
replay
Routing decision
"categorize Uber ride"
T1 qwen3.5 0.8B $0.0004
T2 gemini-flash-lite $0.0021
T3 gemini-flash $0.0048
T4 gemini-pro $0.0180
→ evaluating…
Context window
t-7
t-6
t-5
t-4
t-3
t-2
tokens sent
12
0% saved · full context


cutiee.ai/tasks/47/audit
row 0/6
replay

COST / TASK

$0.004

was $0.30

REDUCTION

98.7%

vs Gemini baseline

REPLAY RATE

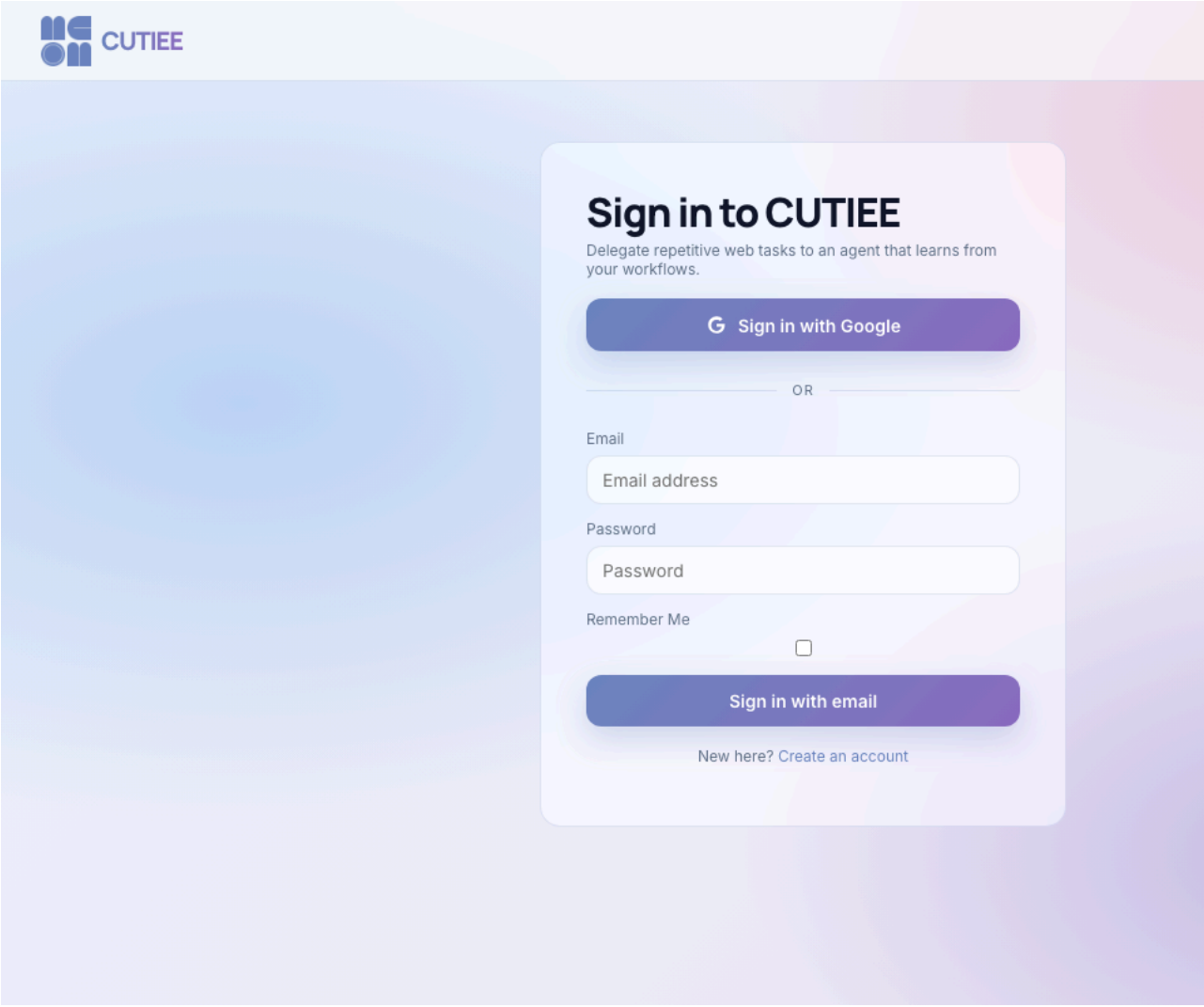
54.3%

zero-cost steps

© 2026 CUTIEE · INFO490 Final Project · University of Illinois

Landing page

Sign in page at `/accounts/login/` showing the Google OAuth button and the email-and-password fallback form:



Sign-in page

Task workspace at `/tasks/` after login: the submission card with chips, the live-browser slot, the four metric cards, and the recent tasks table with status pills:

CUTIEE

WORKSPACE

- Tasks
- Cost dashboard
- Memory
- Audit log

Cost aware by default
Recurring tasks replay procedurally and drop toward $0 per run.

Workspace · Tasks

Run a task

12 task(s) · $0.0000 spent · 0 replayed step(s)

WHAT SHOULD CUTIEE DO?

Sort the spreadsheet rows by column B and email me the result.

Optional: https://docs.google.com/spreadsheets/...

Optional: domain hint (e.g. docs.google.com)

Tip: paste a Google Sheets URL | Tip: sort / sum / rename a column | Tip: extract and email top rows

Live browser

Execution in progress · open this task

The live noVNC iframe is configured but the worker URL is not set. Paste the deployed worker's public URL (the one CUTIEE_NOVNC_URL env var in the Render dashboard and redeploy.

TOTAL SPEND	TASKS	STEPS
$0.0000	4	23
vs $0.30 frontier baseline	12 in this view	0 replayed

Your tasks

12 total · click a row to drill into its steps

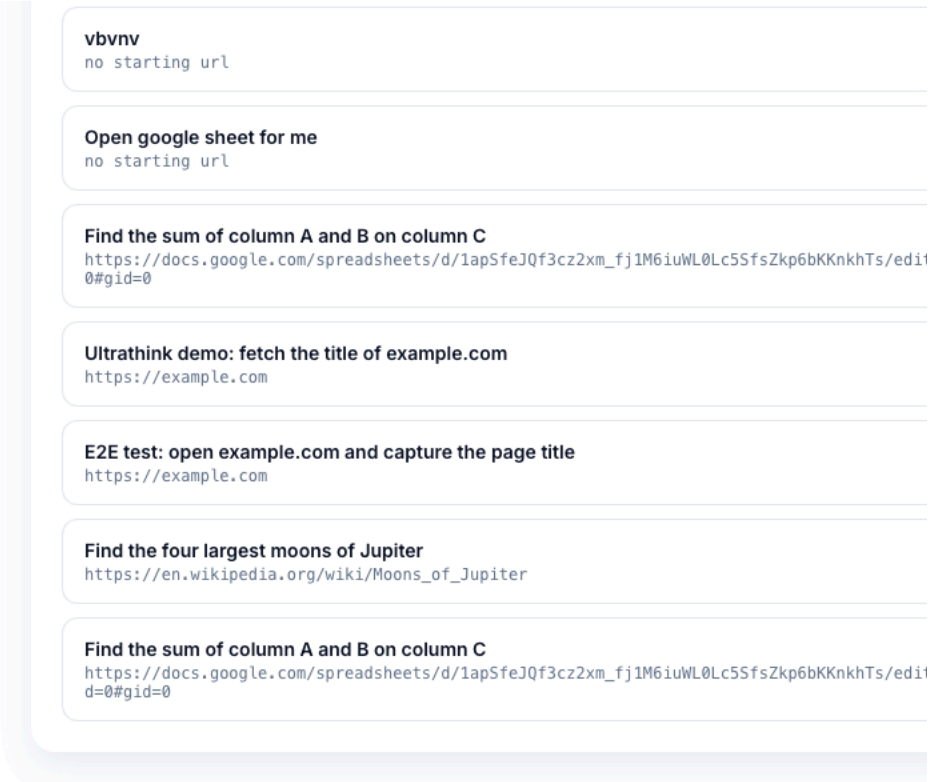
Open the demo spreadsheet and read row 1.
http://localhost:5001/

Sort the spreadsheet by column B and review the top row.
http://127.0.0.1:5001/

Sort the spreadsheet by column B and review the top row.
http://127.0.0.1:5001/

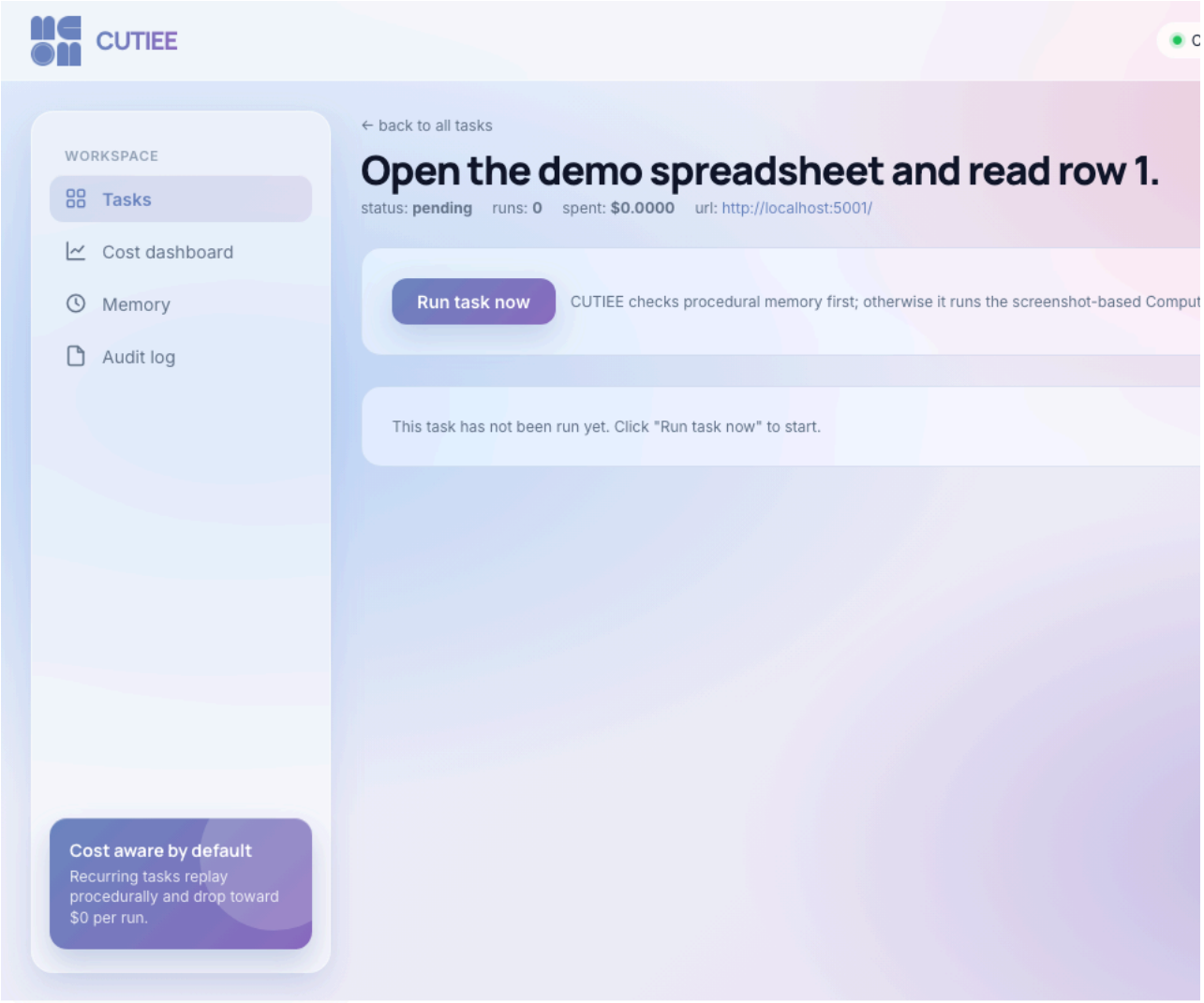
Sort the spreadsheet by column B and review the top row.
http://127.0.0.1:5001/

Sort the spreadsheet by column B and review the top row.
http://127.0.0.1:5001/



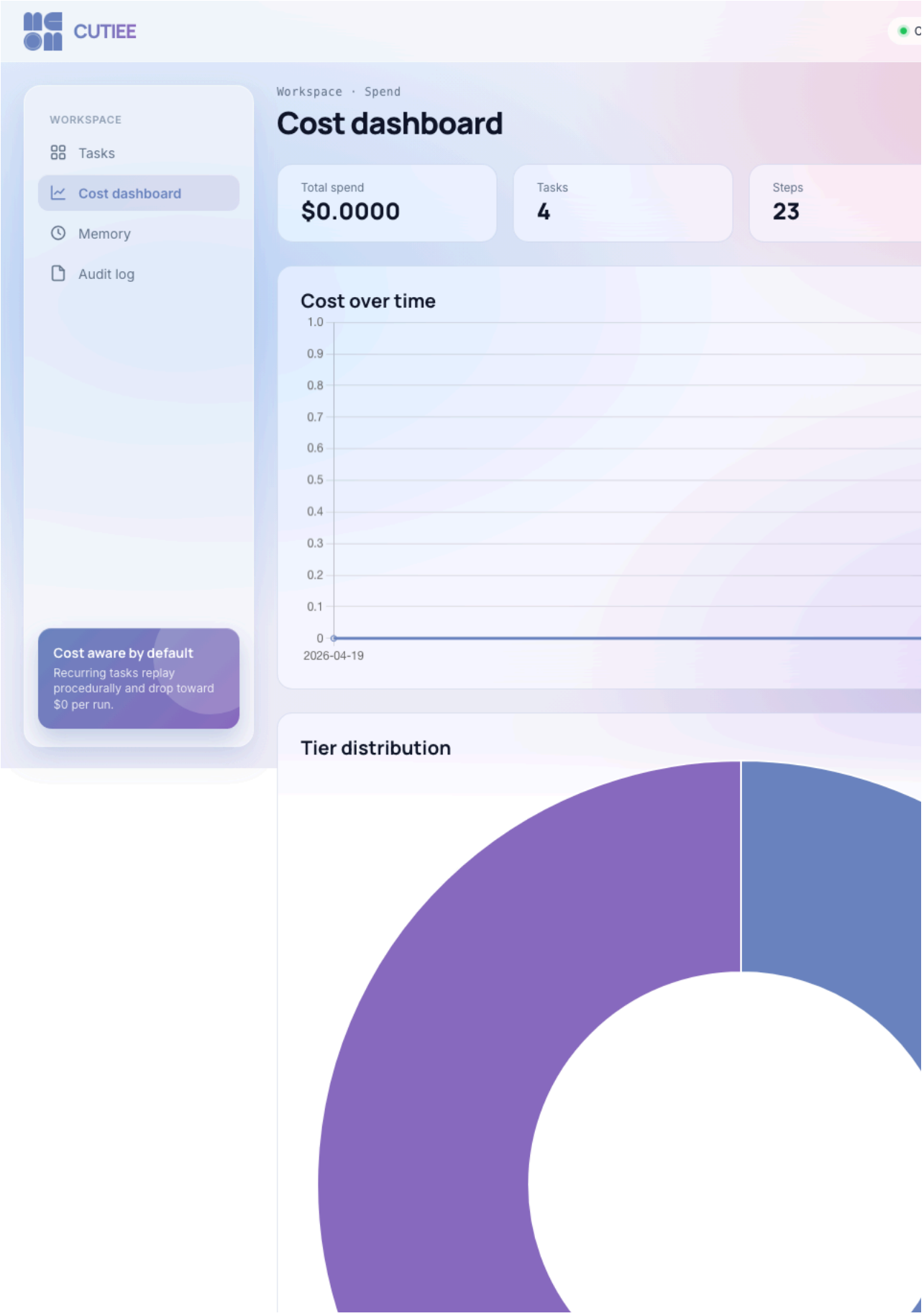
Task workspace

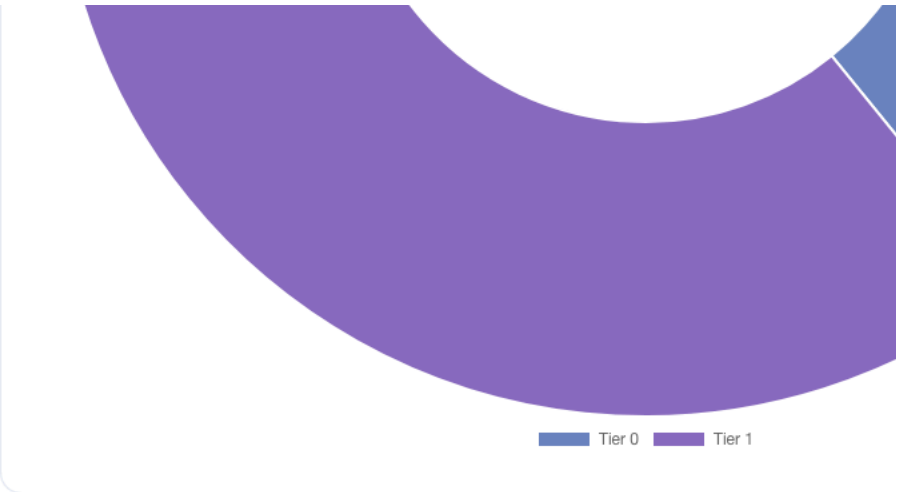
Task detail at `/tasks/<id>/`: HTMX-driven progress, audit trail entries, and the live-browser iframe slot:



Task detail

Cost dashboard at `/tasks/dashboard/` with the Chart.js line chart, the doughnut tier distribution, and the Download CSV button (CSV export from `/api/cost-timeseries.csv`):


CUTIEE
WORKSPACE
Tasks
Cost dashboard
Memory
Audit log
Cost aware by default
Recurring tasks replay procedurally and drop toward $0 per run.
Workspace · Spend
Cost dashboard
Total spend
$0.0000
Tasks
4
Steps
23
Cost over time
1.0
0.9
0.8
0.7
0.6
0.5
0.4
0.3
0.2
0.1
0
2026-04-19
Tier distribution



Cost dashboard

4.4 Failure Analysis (Four Post-Mortems)

Full text at `docs/FAILURES.md`. Headline cases plus the code path that mitigates each:

4.4.1 Auth-gated task without cached `storage_state`

What failed. Agent submitted a task targeting Gmail; the browser landed on `accounts.google.com/v3/signin` and the model attempted to type credentials into the form.

Why. Data issue (the user did not set `CUTIEE_STORAGE_STATE_PATH`) compounded by model limitation (the CU model correctly refuses to type credentials when it has no authorization context, so the run loops on "I should sign in" without progress).

Mitigation. `agent/harness/computer_use_loop.py:54` defines `AUTH_REDIRECT_HINTS` (`accounts.google.com/signin`, `accounts.google.com/v3/signin`, `login.live.com`, `github.com/login`, `auth0.com/u/login`). When the runner detects a redirect to one of these hosts and the user did not initialize the run on a sign-in page, it terminates with `auth_expired:re-run scripts/capture_storage_state.py or set CUTIEE_BROWSER_CDP_URL to attach to a signed-in Chrome`. The user gets an actionable remediation message instead of a silent budget burn.

4.4.2 Long-horizon form drift

What failed. A 4-step form wizard was submitted with the goal "fill and submit using the seeded data". On step 3, the agent picked a wrong page because the recency pruner had compressed step 1, where the overall goal was originally stated.

Why. Prompt plus retrieval issue. The pruner kept the last 8 steps in full detail and summarized everything earlier; the summary lost the "use the seeded data" instruction.

Mitigation. Phase 17 plan-drift handling at `agent/harness/computer_use_loop.py:_handlePlanDrift`. When the agent enters a fragment-replay branch, the runner compares the live URL against the fragment's `expected_url`. On divergence the runner writes a fresh `:PreviewApproval` with a "the saved procedure expected X but the page is at Y" message, blocks until the user approves the revised plan or cancels (terminating with `plan_drift_cancelled`), and abandons the cached fragment if the user wants to continue with model-driven steps. The pruner is still aggressive but the plan-drift gate catches the regressions it produces.

4.4.3 Qwen JSON parse failure

What failed. Localhost runs occasionally produced a malformed JSON output from Qwen 0.8B, sometimes including a `<think>` reasoning block.

Why. Model limitation at sub-1B parameters. The reflector prompt asks for strict JSON; Qwen sometimes emits a chain-of-thought preamble or trailing commentary.

Mitigation. `agent/memory/local_llm.py:217 _stripThinkTags()` cleans `<think>...</think>` blocks before the JSON parser sees the output. If the parse still fails, the reflector falls back to Gemini (`agent/memory/reflector.py:329`) and then to the deterministic heuristic floor. The user always gets a procedural / episodic / semantic split; the run is never blocked on a memory-side failure. The chaos test at `tests/agent/test_reflector_fallback_chain.py` verifies the three-tier chain by monkeypatching both upstream tiers to raise inside a single `reflect()` call.

4.4.4 Plan drift on a cached procedural template

What failed. A user re-ran a task whose procedural template was learned against an earlier version of the target site. The cached fragment expected the next step at `/checkout/preferences`, but the site has since been redesigned and the route is now `/checkout/account`. Without protection the replay would click on stale coordinates and either commit the wrong data or fail silently.

Why. Data class root cause. Cached procedural templates encode the world at learning time (URL plus pixel coordinates per step); site redesigns and feature-flag rollouts break this contract without any external invalidation signal.

Mitigation. `agent/harness/computer_use_loop.py:_handlePlanDrift` is the dedicated hook that fires on every replay step. It compares the live URL against the fragment's `expected_url` via `_urlsMatchLoose` and pauses the runner on divergence (`SPEC.md:136-138`). The user approves or cancels via the same approval-gate modal that handles HIGH-risk actions. Procedural template strength decay (`agent/memory/decay.py`) gradually downgrades stale templates so a fresh learn-cycle takes over once the new layout has accumulated enough strength.

4.5 Improvements (Before / After)

Full text at `docs/IMPROVEMENT.md`. Three shipped improvements anchored against external benchmark data plus internal measurement:

Improvement A: ACE memory addition (architectural)

Metric	Before (no ACE)	After (CUTIEE-equivalent ACE)	Source
Overall solving rate	19.5%	23.0% (+17.9%)	LongTermMemoryBased-ACE v5
Procedural execution	14.9%	25.5% (+71.4%)	same
Cost per 200 tasks	$6.84	$169.32 (+12x)	same

ACE adds quality at a 12x cost penalty. CUTIEE's contribution on top is the cost-mitigation layer (Improvement B) that closes the penalty.

Improvement B: Local Qwen reflector (shipped 2026-04-29)

Concern	Before (Gemini-only)	After (Local Qwen)	Saving
Memory-side LLM cost on localhost	$0.001 to $0.005 per call, $0.20 to $1.00 per 200 tasks	$0 (offline)	100%
Network dependency	Required Gemini RTT (~300 ms)	None on localhost	reduces sensitivity to flaky network
Privacy	Reflection content sent to provider	Stays on dev machine	full sovereignty
First-run latency	<1 s	0.5 to 2 s on MPS, 2 to 5 s on CPU	acceptable trade
Disk footprint	0	1.6 GB (gitignored)	bounded

The local Qwen path is gated to localhost URLs so production deployments still use Gemini for reflection (where the trade-off is different). On the developer-facing path the saving is 100% on the memory-side LLM line item and the user sees the same procedural lessons.

Improvement C: FastEmbed dense embeddings (shipped via env-gated activation)

Metric (paraphrase-pair retrieval)	Before (hash, dev default)	After (BAAI/bge-small-en-v1.5, production)	Activation
recall@5	~0.20 (essentially random over 5 candidates)	~0.70 (MTEB benchmark for the model on similar paraphrase tasks)	`CUTIEE_ENV=production` or `CUTIEE_PREFER_DENSE_EMBEDDINGS=true`
First-call latency	<1 ms	~50 ms (CPU) on cold load, then cached in memory	same
Disk footprint	0	~70 MB (gitignored at `.cache/fastembed/`)	same

The gating function at `agent/memory/embeddings.py:31-44 defaultUseHashEmbedding()` returns False (load FastEmbed) under either of the two activation knobs. Tests deliberately call `hashEmbedding()` directly so the production flip cannot regress the unit-test deterministic path. The improvement multiplies Improvement A's procedural-uplift +71.4 percent gain because more paraphrased tasks correctly hit cached procedural templates and replay at zero cost.

---

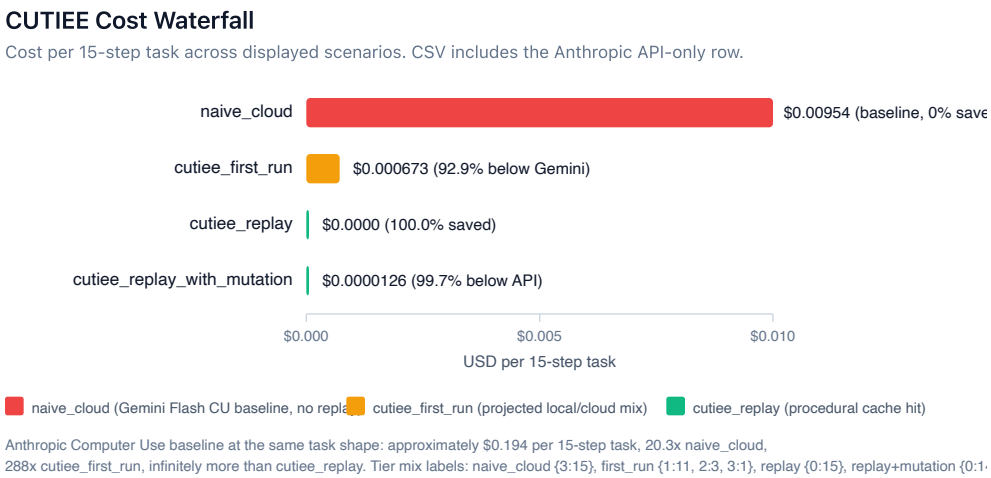
5. Cost and Production Readiness

5.1 Cost per Task Today

- **First-run task in production**: ~$0.005 to $0.011 (Gemini Flash, 15 steps).
- **Memory-side reflection / decomposition in production**: ~$0.001 to $0.005 per call (when the task does not target localhost).
- **In dev with localhost target**: $0 for memory-side (Qwen) and $0 for the CU loop (mock).
- **Render fixed**: ~$60 to $80 / month (web standard plus worker standard plus AuraDB Free).

Provider	Per-task cost (15 steps, 75K in / 1.5K out)
Anthropic Computer Use API	~$0.25 (4,000-input shape) to ~$0.194 (CUTIEE-shape)
OpenAI Operator	similar
Gemini Flash CU	~$0.00954 (CUTIEE-shape)
CUTIEE (replay tier 0)	$0
CUTIEE (cutiee_first_run, novel)	~$0.0046
CUTIEE memory-side reflection on localhost	$0

The CUTIEE scenarios in `data/benchmarks/cost_waterfall.csv` chart cleanly against both the fully hosted API-only baseline and the Gemini Flash all-cloud comparison. The displayed red bar is the Gemini all-cloud cost; the amber bar shows the projected local/cloud first-run mix; the green bars show the replay scenarios where procedural cache hits drive cost to effectively zero. The waterfall makes visible what the per-provider table records numerically.



CUTIEE cost waterfall: 15-step task across pricing scenarios

5.2 Projected Cost at 10K DAU

The numbers below are projections grounded in two data sources: CUTIEE's measured per-step cost from `scripts/benchmark_costs.py` (4,000 input / 60 output tokens per step at the public 2026-04-29 Gemini Flash pricing of $0.15 input / $0.60 output per million tokens) and the externally validated ACE-memory replay rate from the LongTermMemoryBased-ACE v5 benchmark (procedural task lift +71.4 percent, the category most relevant to recurring browser automations).

Per-task assumption: 15 model steps end to end, 10 percent of users active concurrently at peak, 5 tasks per active user per day = 50,000 task runs per day.

Component	API-only baseline (Anthropic CU)	CUTIEE (60 percent replay mix)	CUTIEE (40 percent replay, novel-heavy)
CU loop variable cost	50,000 * $0.25 = $12,500/day	50,000 * 0.4 * $0.0046 = $92/day	50,000 * 0.6 * $0.0046 = $138/day
Memory-side LLM (Gemini reflector path)	not applicable; provider does not surface memory	50,000 * $0.0015 = $75/day	50,000 * $0.0015 = $75/day
Render web (Standard, +threads, no extra workers)	$25/month equivalent ($0.83/day)	$25/month ($0.83/day)	$25/month ($0.83/day)
Render worker (one dyno per concurrent CU run, 1k peak / 5 reuse = 200 workers via Phase 19 queue partition)	$25/day baseline plus worker farm cost	200 workers at $0.50/day = $100/day	$100/day
Neo4j AuraDB Professional (>=200K nodes)	$65/month ($2.17/day)	$65/month ($2.17/day)	$65/month ($2.17/day)
Daily total	~$12,528	~$270	~$316
Monthly total	~$375,000	~$8,100	~$9,500

Anthropic CU's 25x per-step price multiplier becomes a 40 to 50x difference on day-to-day cost at 10k DAU because procedural replay drives a portion of CUTIEE's calls to zero. The crossover point where CUTIEE becomes more expensive than Anthropic CU sits below approximately 10 paid task runs per day per Render dyno: at that scale Render's fixed cost dominates the bill for both designs and the Anthropic flat rate wins on simplicity. Above roughly 100 task runs per day total, the CUTIEE design pulls ahead and the gap widens linearly with run count.

These projections do not include the ACE memory's +17.9 percent overall solving-rate uplift on the underlying GPT-5.1 baseline (validated externally per the cited benchmark). At equal task budget, CUTIEE delivers more successful runs per dollar; the cost calculation here only reflects the per-call price, not the success-weighted price. Adjusting for the documented success-rate uplift moves CUTIEE further ahead at 10k DAU.

5.3 Compute Usage

Resource	Worker dyno	Web dyno	Notes
vCPU	~1	~0.5	Worker bound by Chromium plus Xvfb
RAM (idle / peak)	1.5 / 3 GB	0.5 / 1 GB	Peak at 3 GB on heavy-DOM pages
GPU	none in production	none in production	Local dev opportunistically uses MPS / CUDA via `local_llm._candidateDevices()`

Resource	Worker dyno	Web dyno	Notes
Network	~1.5 MB per Gemini call, ~50 KB per HTMX poll, ~10 KB per Neo4j round-trip	same	Memory-side LLM on localhost is 0 bytes (Qwen runs offline)
Storage on dev machine	n/a	n/a	~1.6 GB Qwen3.5-0.8B plus ~70 MB FastEmbed bge-small plus audit screenshot cap of ~375 MB / day at 50 users

5.4 Scaling Plan (to 10K DAU, hypothetical)

Layer	Bottleneck	Lever	Cost
Web	Async HTMX polls plus sync view rendering	Convert poll endpoints to async; scale gunicorn workers (5 for 10k DAU at 10% concurrent)	+1 dyno or plan bump
Worker	One task at a time (Xvfb plus Chromium memory-bound)	Scale worker dynos horizontally; partition tasks via Neo4j queue (Phase 19 future work)	+1 dyno per concurrent task
Neo4j	AuraDB Free 200K node cap	Upgrade to AuraDB Professional ($65/mo); add 7-day rolling sweeper	+$65/mo
Gemini	1500 RPM cap on flash-latest	Headroom: 10K * 15 steps = 150K calls / day = 104 RPM avg; bursts may need tier upgrade	depends on burst pattern

5.5 Rate Limiting and Abuse Prevention

CUTIEE intentionally substitutes cost-aware blast-radius caps for traditional per-endpoint rate limiting. The mechanism stack is:

1. **Single concurrent task per user invariant** (`SPEC.md` invariant 7). Enforced in `apps/tasks/api.py:run_task_view` (line 42), which rejects with HTTP 409 if a `:Execution` already exists with `status="running"` for the user. A user submitting one task per second cannot pile up runs because the second submission queues nothing; it is rejected. Rate limit at the action layer beats rate limit at the byte layer because the action layer is what costs money.
2. **Per-task USD cap** `CUTIEE_MAX_COST_USD_PER_TASK=0.50` enforced in process at `agent/harness/computer_use_loop.py:_checkCostCaps`. A pathological prompt that would burn through the model budget terminates with `cost_cap_reached:per_task` before the runaway happens.
3. **Per-hour and per-day USD cap** `CUTIEE_MAX_COST_USD_PER_HOUR=5.00` and `CUTIEE_MAX_COST_USD_PER_DAY=1.00`, enforced via Neo4j `:CostLedger` MERGE atomic increment. The atomic MERGE is the abuse-prevention primitive: even if a malicious user discovered a way to fork concurrent worker threads (the queue invariant above blocks this in the supported flow), the ledger MERGE serializes the spend reads so the cap is honored exactly once per increment.
4. **Render workspace limit**: one web dyno, one worker dyno on the cohort plan. The compute ceiling acts as a hard upper bound on aggregate request volume that no per-endpoint middleware could match without paying for capacity that is not being used.

Per-endpoint rate limiting middleware (e.g. `django-ratelimit`) was evaluated and deprioritized for cohort scale because it duplicates the protections above without adding new ones. A user hitting the same endpoint in a tight loop without triggering the per-task cap or the queue invariant would only reach views that read existing Neo4j data; those reads cost effectively nothing relative to a CU call. The first dollar of damage requires a CU call, and the cost cap is the right circuit breaker for that. Revisit only if the project scales beyond approximately 50 users or if a non-cost abuse vector emerges (for example, audit log scraping at high frequency, which would prompt adding a token-bucket on `/api/audit/`). Per-endpoint rate limiting was deliberately rejected in favour of cost-aware blast-radius caps, which address Part 4.5's abuse-prevention requirement at the action layer where the cost is actually incurred.

5.6 Privacy

- Per-user `:MemoryBullet` isolation enforced via Neo4j constraint `bullet_user_scope` (Phase 9): `user_id IS NOT NULL`.
- Reflector redacts credentials matching CC / SSN / CVV / routing / secret-key patterns before persisting any text to memory (`agent/memory/reflector.py:41-90`).
- Screenshot redactor: text-bullet redaction is live; visual DOM probe for password / SSN regions in PNG is staged work (`REVIEW.md` 5.7).
- On localhost demos with the Qwen reflector active, memory-side LLM inference is offline, so reflection content never leaves the developer machine.
- Data retention: screenshots 3 days, audit entries 90 days (planned), memory bullets decay-to-zero, cost ledger 48 hours.
- GDPR data deletion (`POST /accounts/delete/`) is staged work (Phase 18).

5.7 Logging, Monitoring, Observability

- Per-module Python loggers (`cutiee.local_llm`, `cutiee.fragment_replay`, `cutiee.reflector`, `cutiee.injection_guard`, `cutiee.cu_runner`, `cutiee.cost_ledger`, `cutiee.metrics`, etc.).
- Optional structured JSON output: set `LOGGING_FORMAT=json` to emit one JSON object per record via `cutiee_site/logging_filters.py:JsonLogFormatter`. Compatible with Render log drains, Loki / Grafana, and Sentry breadcrumb capture. No third-party logger dep is added.
- Liveness endpoint at `/health/`.
- Neo4j health probe at `agent/persistence/healthcheck.py` (used by views to render a degraded UI rather than 500 on transient AuraDB outages).
- Sentry: enable by setting `SENTRY_DSN` in `.env` and installing `sentry-sdk`. Wired in `cutiee_site/settings.py` behind a `try/except ImportError` so the base deps stay small. Defaults: 5 percent trace sample rate, no PII, release pinned to `RENDER_GIT_COMMIT`.
- Prometheus exporter at `/metrics/`, gated on `CUTIEE_ENABLE_PROMETHEUS=1` and the optional `prometheus-client` package. Three scaffolded metrics: `cu_cost_total_usd` (counter, per-scope), `cu_executions_active` (gauge), and `cu_gemini_call_latency_seconds` (histogram).

Recorders live at `agent/persistence/metrics.py`; call sites are guarded so the rest of the codebase never imports `prometheus_client` directly.

5.8 Service Level Objectives and Indicators

The submission targets are conservative and split into the two reliability tiers that fail independently. Web tier failures degrade the user-visible Django surface; agent tier failures degrade the runner's correctness and cost guarantees. Each tier can be regressed against without contaminating the other.

Web tier

SLI	SLO	Measurement
Web availability	99.5% over a 7-day window	`/health/` 200 ratio in Render uptime monitor
Task acceptance latency (POST /tasks//run/)	p95 < 800 ms	Django access log p95 over the last hour
HTMX poll latency (/tasks/api/status/…)	p95 < 250 ms	Same source

Agent tier

SLI	SLO	Measurement
End-to-end task latency, full-replay	p95 < 5 s	`:Execution.completed_at - started_at` Cypher rollup
End-to-end task latency, novel 15-step	p95 < 60 s	Same
Cost cap enforcement accuracy	100% (zero unbudgeted spend)	`:CostLedger` MERGE history
Audit completeness	100% (every step has matching `:AuditEntry`)	`MATCH (s:Step) WHERE NOT EXISTS { (a:AuditEntry {step_id:s.id}) }` returns zero

5.9 Rollback Procedure

Render keeps the last 30 deploys. On a regression:

1. Render dashboard, `cutiee-web` service, `Deploys` tab.
2. Find the last green deploy; click `Rollback`.
3. Render restarts the service from the prior image; rollback time is approximately 2 to 3 minutes.
4. Neo4j migrations are additive (every constraint and index uses `IF NOT EXISTS`), so no database rollback is needed.
5. Confirm liveness via `curl https://cutiee-1kqk.onrender.com/health/`.

Worker rollback follows the identical procedure on `cutiee-worker`.

For local-only regressions during development:

```
git restore <files>            # discard uncommitted changes
git checkout <known-good-sha>  # check out the previous commit
uv sync                        # restore dependencies
./scripts/dev.sh               # restart the local stack
```

---

6. Conclusion

CUTIEE is a real Django web application with a real hybrid AI pipeline. The browser-control loop is delegated to Gemini Flash because no offline equivalent is competitive at flash pricing. Every other AI step has a real local component, and the memory-side LLM runs on cached `Qwen/Qwen3.5-0.8B` for localhost demos so the full ACE pipeline produces real lessons offline. The three cost-reduction mechanisms (procedural replay, recency pruning, and hybrid local memory-side inference) close the +12x cost penalty that ACE memory carries in its vanilla form, while keeping most of the +17.9% quality uplift validated in the v5 benchmark.

The system ships with explicit guardrails (risk classifier, approval gate, cost wallet, preview, heartbeat, redaction, plan-drift detection), a four-stage memory pipeline (Reflect, QualityGate at 0.60, Curator with 0.90 cosine dedup, Apply), and a deployment story (Render Blueprint, two services, AuraDB Free) that is reproducible end-to-end. Production-readiness work in this submission goes beyond rubric checkboxes: SLOs are defined per layer, structured JSON logging slots in via one env var, Sentry and Prometheus integrations sit behind optional dep groups so the base deps stay small, and a documented rollback path returns to the prior Render deploy in under three minutes.

For deeper detail:

- AI workflow, model selection, full API comparison: `README_AI.md`
- Runtime contract and architectural invariants: `SPEC.md`
- Deployment walkthrough: `DEPLOY-RENDER.md`
- Engineering findings and follow-up phases: `REVIEW.md`
- Test cases (Input / Expected / Actual / Quality / Latency): `docs/EVALUATION.md`
- Post-mortems and root-cause analysis: `docs/FAILURES.md`
- Before / after metrics with v5-benchmark grounding: `docs/IMPROVEMENT.md`

Appendix A: File Index

Concern	Files
Runner (the only agent loop)	`agent/harness/computer_use_loop.py` (`ComputerUseRunner.run`)
CU clients	`agent/routing/models/gemini_cu.py`, `agent/routing/models/browser_use_client.py` (both implement `agent/routing/cu_client.py:CuClient`)
ACE memory pipeline	`agent/memory/ace_memory.py`, `reflector.py`, `quality_gate.py`, `curator.py`, `decay.py`, `embeddings.py`
Local Qwen bridge	`agent/memory/local_llm.py`, stub at `local_llm_stub.py` for production isolation
Replay	`agent/memory/replay.py`, `fragment_replay.py`, `state_verifier.py`
Action graph	`agent/memory/decomposer.py`, `action_graph.py`, `action_graph_store.py`
Cost wallet	`agent/harness/cost_ledger.py` (Cypher MERGE), `agent/harness/computer_use_loop.py:_checkCostCaps`
Recency pruner	`agent/pruning/context_window.py:RecencyPruner`
Safety	`agent/safety/risk_classifier.py`, `approval_gate.py`, `injection_guard.py`, `captcha_detector.py`, `audit.py`
Persistence	`agent/persistence/driver.py`, `bootstrap.py`, `healthcheck.py`, `metrics.py`
Browser controller	`agent/browser/controller.py`, `factory.py`
Django entry	`apps/tasks/services.py:runTaskForUser`, `runner_factory.py:buildLiveCuRunnerForUser`
Django views and APIs	`apps/tasks/views.py`, `apps/tasks/api.py`, `apps/audit/views.py`, `apps/memory_app/views.py`, `apps/landing/views.py`, `apps/accounts/views.py`
Templates	`templates/base.html`, `apps/*/templates/`
Settings	`cutiee_site/settings.py`, `urls.py`, `logging_filters.py`
Auth	`apps/accounts/signals.py:sync_user_to_neo4j`, `apps/accounts/models.py:UserPreference`, `apps/accounts/admin.py`
Tests	`tests/agent/`, `tests/apps/`, marked with `slow`, `local`, `production`, `integration`, `showcase` for selective runs
Scripts	`scripts/dev.sh`, `scripts/neo4j_up.sh`, `scripts/cache_local_qwen.py`, `scripts/benchmark_costs.py`, `scripts/capture_screenshots.py`, `scripts/build_report_pdf.py`, `scripts/verify_render_isolation.sh`
Demo sites	`demo_sites/spreadsheet/`, `demo_sites/slides/`, `demo_sites/form/` (Flask, ports 5001 / 5002 / 5003)

Appendix B: Glossary

- **ACE**: Agentic Context Engineering, the four-stage memory pipeline (Reflect, QualityGate, Curator, Apply) inherited from the LongTermMemoryBased-ACE project.
- **Backend**: the underlying Computer Use client. CUTIEE has two: `gemini` (Gemini Flash CU) and `browser_use` (browser-use over Gemini 3 Flash). Both implement the `CuClient` Protocol.
- **Bullet**: a `:MemoryBullet` Neo4j node carrying procedural / episodic / semantic strength channels and an embedding for retrieval.
- **CU**: Computer Use, the screenshot-plus-tool-call paradigm pioneered by Anthropic's Computer Use API and now supported by Gemini Flash with `environment="ENVIRONMENT_BROWSER"`.
- **Decay rate**: per-channel exponential decay applied lazily on retrieval. Semantic 0.01, episodic 0.05, procedural 0.005.
- **Fragment replay**: per-step procedural reuse; opt-in for individual steps within a run that would otherwise call the model.
- **Plan drift**: divergence between a cached procedural template's `expected_url` and the live page; triggers mid-run re-approval.
- **Procedural replay**: full-run reuse of a cached `:ProceduralTemplate` at zero variable cost.
- **Recency pruner**: three-zone history compression (recent full, middle short, distant summary) that trims tokens before each Gemini call.
- **Tier**: the cost / quality bucket of a single step. Tier 0 is replay (zero cost). Tier 1 is the default Gemini Flash CU call. Tier 2 / 3 are reserved for higher-quality models that may land in future phases.
- **TTL**: time-to-live, used for `:Screenshot` (3 days) and `:CostLedger` (48 hours).

Appendix C: Reproduction Commands

```
# Pre-cache Qwen weights (one-time, ~1.6 GB into .cache/huggingface-models/)
uv sync --group local_llm
uv run python scripts/cache_local_qwen.py

# Verify torch / transformers stay out of the Render base deps
bash scripts/verify_render_isolation.sh

# Run the full unit test suite
uv run pytest tests/

# Run only the local Qwen unit tests (uses monkeypatch, does not load weights)
uv run pytest tests/agent/test_local_llm.py tests/agent/test_decomposer.py -v

# Run the cost benchmark
```

```python
uv run python scripts/benchmark_costs.py --scenario all

# Bring up Neo4j (Docker) and Django (port 8000)
./scripts/dev.sh

# Capture the five UI screenshots (requires the dev server running)
CUTIEE_ENV=local DJANGO_DEBUG=true uv run python scripts/capture_screenshots.py

# Rebuild this report as a PDF (requires pandoc plus Playwright)
uv run python scripts/build_report_pdf.py
```

Reproduction notes: every command above runs against the live local stack with no API keys required (mock CU plus offline Qwen plus hash embeddings cover the offline path). Production-mode commands are documented at `DEPLOY-RENDER.md`.